

main.c

Share

Run

Output

Clear

```
2  #include <string.h>
3
4  int main() {
5      char text[100];
6      int a, b, i;
7
8      printf("Enter plaintext: ");
9      scanf("%s", text);
10
11     printf("Enter a and b: ");
12     scanf("%d %d", &a, &b);
13
14     printf("Ciphertext: ");
15
16     for (i = 0; text[i] != '\0'; i++) {
17         int p = text[i] - 'A';
18         int c = (a * p + b) % 26;
19         printf("%c", c + 'A');
20     }
21
22     return 0;
23 }
```

Enter plaintext: ABC
Enter a and b: 5 8
Ciphertext: INS

=== Code Execution Successful ===



main.c



```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 uint32_t F(uint32_t r, uint32_t key)
5 {
6     return r ^ key;
7 }
8
9 uint64_t DES(uint64_t data, uint32_t key)
10 {
11     uint32_t L = data >> 32;
12     uint32_t R = data & 0xFFFFFFFF;
13
14     for (int i = 0; i < 16; i++)
15     {
16         uint32_t temp = R;
17         R = L ^ F(R, key);
18         L = temp;
19     }
20
21     return ((uint64_t)R << 32) | L;
22 }
```

Output

Clear

Plaintext : 123456789ABCDEF0
Ciphertext : 9BBCDFF19ABCDEF0
Decrypted : 123456789ABCDEF0

=== Code Execution Successful ===

main.c

[] [] Share Run

Clear

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int main() {
6     int message = 123;
7     int privateKey = 7;
8
9     // DSA-like demonstration
10    srand(time(NULL));
11
12    int k1 = rand() % 100 + 1;
13    int k2 = rand() % 100 + 1;
14
15    int dsa_sig1 = (message + k1 * privateKey) % 1000;
16    int dsa_sig2 = (message + k2 * privateKey) % 1000;
17
18    // RSA-like deterministic demonstration
19    int rsa_sig1 = (message * privateKey) % 1000;
20    int rsa_sig2 = (message * privateKey) % 1000;
21
22    printf("DSA Signature 1 = %d\n", dsa_sig1);
```

DSA Signature 1 = 389
DSA Signature 2 = 466
RSA Signature 1 = 861
RSA Signature 2 = 861

=== Code Execution Successful ===

main.c

[] [] Share Run

```
27- if(inv==26) {  
28     printf("Attack failed: plaintext matrix has no  
        inverse.\n");  
29     return 0;  
30 }  
  
31 /* K = C * P^-1 mod 26 */  
32 K[0][0] = mod26((C[0][0]*P[1][1] -  
33                C[0][1]*P[1][0]) * inv);  
34 K[0][1] = mod26((-C[0][0]*P[0][1] +  
35                C[0][1]*P[0][0]) * inv);  
36 K[1][0] = mod26((C[1][0]*P[1][1] -  
37                C[1][1]*P[1][0]) * inv);  
38 K[1][1] = mod26((-C[1][0]*P[0][1] +  
39                C[1][1]*P[0][0]) * inv);  
40  
41  
42 printf("\nRecovered Key Matrix:\n");  
43 printf("%d %d\n%d %d\n", K[0][0],K[0][1],  
44          K[1][0],K[1][1]);  
45  
46 return 0;  
47 }
```

Output

Clear

Enter 2x2 plaintext matrix:
1 0
0 1
Enter corresponding ciphertext matrix:
5 9
3 0

Recovered Key Matrix:
5 9
3 0

=== Code Execution Successful ===

main.c

[Full Screen] [Theme] Share Run Output Clear

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char text[500];
6     char freq[] = "ETAOINSHRDLCLUMWFGYPBVKJXQZ";
7     char map[26];
8     int count[26] = {0}, used[26] = {0};
9
10    printf("Enter ciphertext: ");
11    fgets(text, sizeof(text), stdin);
12
13    /* Count letter frequencies */
14    for (int i = 0; text[i]; i++)
15        if (text[i] >= 'A' && text[i] <= 'Z')
16            count[text[i] - 'A']++;
17
18    /* Create frequency-based substitution */
19    for (int k = 0; k < 26; k++) {
20        int max = -1, pos = -1;
21
22        for (int i = 0; i < 26; i++)
```

```
Enter ciphertext: E T A O I N S H R D L C U M W F G Y P B V K J X Q Z

Possible plaintext:
I B E W R M P H Y O C A V U J N S Q F T K L D X G Z

Top 10 possible plaintexts can be generated by trying small variations of the frequency mapping.

=== Code Execution Successful ===
```

[illegible]

main.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define BLOCK 4
5
6 void pad(char *data, int *len) {
7     data[(*len)++] = 0x80;        // 1 bit followed by zeros
8     while (*len % BLOCK != 0)
9         data[(*len)++] = 0x00;
10 }
11
12 void ECB(char *data, int len) {
13     for (int i = 0; i < len; i += BLOCK)
14         for (int j = 0; j < BLOCK; j++)
15             data[i+j] ^= 0xAA;
16 }
17
18 void CBC(char *data, int len) {
19     char IV = 0x55;
20
21     for (int i = 0; i < len; i += BLOCK) {
22         for (int j = 0; j < BLOCK; j++) {
```

Output

Padded length = 8 bytes
ECB encryption completed.
CBC encryption completed.
CFB encryption completed.

=== Code Execution Successful ===

Stop copy pasting code you don't actually understand

Build the coding confidence you need to become a developer companies will fight for

Become a PRO

Programiz

C Online Compiler

BitRaser®
Drive Eraser

NIST Tested and Approved
Drive Wiping Software

stellar

REQUEST A FREE DEMO



main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 uint64_t generate_subkey(uint64_t L, uint64_t Rb)
5 {
6     uint64_t msb = L >> 63;
7     L <<= 1;
8
9     if (msb)
10         L ^= Rb;
11
12     return L;
13 }
14
15 int main()
16 {
17     uint64_t L = 0x8000000000000000ULL;
18
19     uint64_t K1 = generate_subkey(L, 0x1B);
20     uint64_t K2 = generate_subkey(K1, 0x1B);
21
22     printf("K1 = %016llx\n", K1);
```

```
K1 = 000000000000001B
K2 = 0000000000000036

=== Code Execution Successful ===
```